



KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Hafta 3 Laboratuvar Föyü

JavaScript'ten TypeScript'e: Hafta 2 Sayfasını Canlandırın

Doç. Dr. Tolga Berber · <https://avesis.ktu.edu.tr/tberber>

- 🕒 **Tahmini süre:** 2 saat
- 🖨 **Gereksinimler:** Node.js 24 LTS kurulu bir bilgisayar (node --version ile doğrulayın), bir metin editörü (VS Code önerilir), güncel bir tarayıcı (Chrome/Firefox/Edge), Hafta 2'de tamamladığınız web-lab klasörü — npm paketlerini indirmek için internet bağlantısı gerekir

Öğrenme fazları: 1 • Ön-öğretim 2 • İçerik 3 • Katılım 4 • Değerlendirme 5 • İzleme

1 • Ön-öğretim 📣 Neden Önemli?

Hafta 2'de kurduğunuz koltuk planındaki koltuklara tıklayınca hiçbir şey olmuyordu; bilet formunda “Gönder”e basınca sayfa boşuna yenileniyordu. O sayfa tamamen **cansızdı** — tarayıcı onu diskten okuyup olduğu gibi çiziyordu, başka bir şey yapmıyordu. Bugün iki saat sonra aynı header/nav/main/footer iskeletine **tek satır bile dokunmadan**, üstüne yazdığınız TypeScript ile o koltuklar tıklanabilir olacak, seçili koltuk sayısı ve toplam tutar anlık güncellenecek, form gerçekten “gönderilecek”. Bu, Etkinlik/Bilet Sisteminin ilk **davranış** katmanı — ve JavaScript/TypeScript'i C++'tan tanıdığınız değer/referans, fonksiyon ve tip kavramlarıyla köprüleyerek öğreneceksiniz.

1 • Ön-öğretim 🔄 Ön Bilgi

Bu föy Hafta 3 ders notunun dört ünitesine dayanır: Ünite 1 (JavaScript dil modeli: değer/referans semantiği, fonksiyonlar, closure, nesne/prototip), Ünite 2 (modüller ve araç zinciri: ES modülleri, npm, Vite), Ünite 3 (TypeScript tip sistemi: interface, union, readonly, generic) ve Ünite 4 (asenكرون JavaScript ve DOM: olay döngüsü, Promise, async/await). YG I/II'den C/C++'ın değer/referans/pointer ayrımını, kendi `Liste<T>` şablon sınıfınızı ve `std::thread` ile gerçek paralel iş parçacığı deneyiminizi hazır bilgi olarak kullanacaksınız — her yeni JS/TS kavramı bu bilgiyle köprülenecek. Hafta 2'de yazdığınız `index.html`, `etkinlik-detay.html` ve `still.css` dosyalarının iskeleti bu hafta **değişmeden** korunur; React henüz yoktur (Hf. 4'ün konusudur).

Föyü şöyle kullanın: her adımda önce *Ne yapacağız?* kutusundaki görevi kendi dosyalarınıza **elle yazarak** tamamlayın, sonra terminalde/tarayıcıda gördüğünüzü *Beklenen sonuç* kutusuyla karşılaştırın. Adımlardaki kod kutuları `kod_ornekleri/hafta03/` klasöründeki referans dosyalardan gelir — bunlar hedef/“doğru cevap” niteliğindedir; siz bu kodu kopyala-yapıştır yapmak yerine kendi klasörünüzdeki dosyalara yazarsınız, sonucu terminalde/tarayıcıda karşılaştırırsınız. Adımlar ders notunun dört ünitesiyle aynı sırada ilerler; her bölümün başında ilgili üniteye bir atıf bulacaksınız. Birkaç adımda bir gelen *Kontrol Noktası*, o ana kadar yazdıklarınızı bir arada toparlamanıza yarar.

1 JavaScript Dil Modeli: Değer/Referans, Fonksiyonlar, Closure

(bkz. Ders notu Hafta 3, Ünite 1 — JavaScript Dil Modeli: Değer/Referans, Fonksiyonlar, Closure, Nesne ve Prototip.)

Adım 1: js-lab klasörünüzü ve boş dosyalarınızı hazırlayın

? Ne yapacağız? Web-lab klasörünüzün yanında js-lab adlı yeni bir klasör açıp içine beş boş dosya — `deger-referans.js`, `closure.js`, `prototip-sinif.js`, `modul-disa-aktar.mjs`, `modul-ice-aktar.mjs` — oluşturacaksınız.

i Neden? Bu ünitenin tüm örnekleri npm/Vite gerektirmez — Node.js çalışma zamanı bir `.js/.mjs` dosyasını doğrudan çalıştırabilir (Ünite 2'de bu “çalışma zamanı” kavramını derinleştireceksiniz). Beş dosyayı en baştan hazırlamak, ilerleyen adımlarda doğru kodu doğru dosyaya yazdığınızdan emin olmanızı sağlar.

_ Terminal

```
$ mkdir js-lab && cd js-lab
$ touch deger-referans.js closure.js prototip-sinif.js
$ touch modul-disa-aktar.mjs modul-ice-aktar.mjs
$ node --version
```

👁 Beklenen sonuç: js-lab klasöründe beş boş dosya görünür; `node --version` çıktısı v24 ile başlar (LTS) — daha yeni bir Current sürümü kuruluysa (ör. v26) bu haftanın örnekleri yine de çalışır, ama ders boyunca kararlılık için Node 24 LTS önerilir.

🖥 Beklenen Çıktı

```
closure.js
deger-referans.js
modul-disa-aktar.mjs
modul-ice-aktar.mjs
prototip-sinif.js
```

🖥 macOS / Linux

Yukarıdaki `mkdir` ve `touch` komutları her iki sistemde de hazır gelir.

🖥 Windows

PowerShell'de sırasıyla:

- `mkdir js-lab; cd js-lab`
- `ni deger-referans.js, closure.js, prototip-sinif.js`
- `ni modul-disa-aktar.mjs, modul-ice-aktar.mjs`

komutlarıyla aynı beş dosyayı oluşturun.

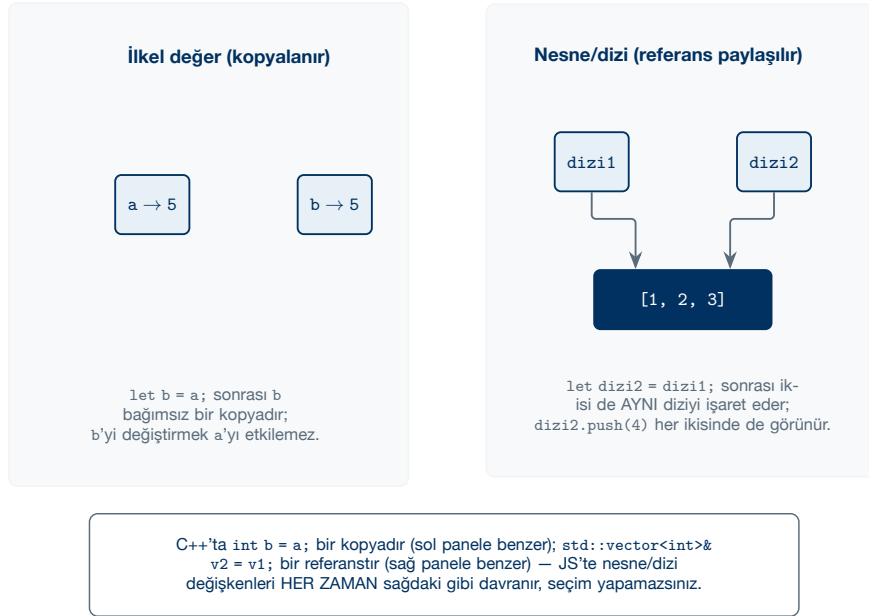
Adım 2: Değer semantiği ile referans semantiğini karşılaştırın

? Ne yapacağız? `deger-referans.js` dosyanıza aşağıdaki kodu elle yazacaksınız: bir `number` değişkenini başka bir değişkene atayıp değiştirecek, ardından bir dizi değişkenini başka bir değişkene atayıp değiştirecek, son olarak diziyi bir fonksiyona parametre olarak

Adım 2 (devamı)

geçirip fonksiyon içinde değiştireceksiniz.

i Neden? C++'ta `int b = a;` her zaman bir kopyadır; referans istiyorsanız açıkça `&` yazmanız gerekir. JavaScript'te ilkel tipler (`number`, `string`, `boolean`) hep kopyalanarak atanır — ama nesne ve dizi değişkenleri her zaman ortak belleğe işaret eden bir referans taşır, “kopya” seçeneğiniz yoktur. Bu ayrımı görmeden yazılan kod, beklenmedik yerlerde “gizemli” mutasyonlara yol açar.



```
1 // İlkel değer: kopyalanarak atanır (C++'taki "int b = a;" gibi bir kopya).
2 let a = 5;
3 let b = a;
4 b = 10;
5 console.log("a:", a); // 5 – b'nin değişmesi a'yı etkilemez
6
7 // Nesne/dizi: değişken her zaman ortak belleğe işaret eden bir referans
8 // taşır (C++'taki T&/pointer semantiğine benzer; "kopya" seçeneği yoktur).
9 let dizi1 = [1, 2, 3];
10 let dizi2 = dizi1;
11 dizi2.push(4);
12 console.log("dizi1:", dizi1); // [1, 2, 3, 4] – dizi2 ile AYNI diziyi işaret eder
13
14 // Aynı fikir fonksiyon parametresinde de geçerlidir: dizi bir referans
15 // olarak geçirilir, fonksiyon içindeki değişiklik dışarıdan da görünür.
16 function ekle(dizi) {
17   dizi.push(99);
18 }
19 let sayilar = [1, 2, 3];
20 ekle(sayilar);
21 console.log("sayilar:", sayilar); // [1, 2, 3, 99]
```

>_ Terminal

\$ node deger-referans.js

👁 Beklenen sonuç: Üç `console.log` satırı sırasıyla `a`'nın değişmediğini, ama `dizi1` ve `sayilar`'ın — her ikisi de bir referans paylaştığı için — değiştiğini gösterir.

Adım 2 (devamı)

Beklenen Çıktı

```
a: 5
dizi1: [ 1, 2, 3, 4 ]
sayılar: [ 1, 2, 3, 99 ]
```

Ortak Hata / Tuzak

let dizi2 = dizi1; satırının dizi1'in bir **kopyasını** oluşturduğunu düşünmek en sık yapılan hatadır. Aslında dizi2 yalnızca dizi1 ile aynı diziye giden ikinci bir isimdir — bağımsız bir kopya istiyorsanız yayma (spread) operatörünü ([...dizi1]) kullanmanız gerekir.

Adım 3: Closure ile bağımsız bir sayaç oluşturucu yazın

❓ Ne yapacağız? closure.js dosyanıza, çağrıldığında kendi bağımsız değer değişkenini tutan bir iç fonksiyon döndüren olusturSayac() fonksiyonunu yazacak, ardından bu fonksiyondan iki bağımsız sayaç (sayac1, sayac2) oluşturup sırayla çağıracaksınız.

❗ Neden? olusturSayac() çalışmasını bitirip “kapandıktan” sonra bile, döndürdüğü iç fonksiyon kendi değer değişkenine erişmeye devam eder — bu, closure'ın tanımıdır. C++'ta buna en yakın şey bir nesnenin (ör. sayac) üye değişkeninde durumu saklamasıdır; JavaScript'te aynı işi bir fonksiyonun kendi kapsamında saklayarak yaparsınız, ayrı bir sınıf tanımlamanıza gerek kalmaz.

```
1 // olusturSayac(), her çağrıldığında kendi bağımsız "değer" değişkenine
2 // sahip YENİ bir fonksiyon döndürür. Dönen iç fonksiyon, dış fonksiyon
3 // (olusturSayac) çoktan çalışmasını bitirmiş olsa bile "değer"e erişmeye
4 // devam eder — bu, closure'ın tanımıdır.
5 function olusturSayac() {
6   let deger = 0;
7   return function () {
8     deger++;
9     return deger;
10  };
11 }
12
13 const sayac1 = olusturSayac();
14 const sayac2 = olusturSayac();
15
16 console.log("sayac1():", sayac1()); // 1
17 console.log("sayac1():", sayac1()); // 2
18 console.log("sayac2():", sayac2()); // 1 — sayac1'den bağımsız kendi değer'i
19 console.log("sayac1():", sayac1()); // 3
```

> Terminal

```
$ node closure.js
```

👁 Beklenen sonuç: sayac2() kendi bağımsız sayacını 1'den başlatır — sayac1'in o ana kadar üç kez çağrılmış olması sayac2'yi hiç etkilemez.

Adım 3 (devamı)**Beklenen Çıktı**

```
sayac1(): 1
sayac1(): 2
sayac2(): 1
sayac1(): 3
```

Adım 4: Prototip zincirini elle kurup class söz dizimiyle karşılaştıran

? Ne yapacağız? prototip-sinif.js dosyanıza önce `Object.create` ile bir prototip nesnesine elle bağlı bir katilimci oluşturacak, ardından aynı davranışı ES6 class söz dizimiyle (Katilimci sınıfı ve katilimci2 örneği) yazacaksınız.

i Neden? Bir nesnede bulunmayan bir özellik (ör. `selamla`) istendiğinde, JavaScript motoru bu özelliği önce nesnenin kendi prototipinde, o da yoksa onun prototipinde arar — class söz dizimi bu mekanizmanın üstüne kurulmuş, okunaklı bir kısayoldur ("syntactic sugar"); class ile tanımladığınız metotlar da aslında prototipte durur. C++'ta sınıf üyeleri her örnekte ayrı ayrı saklanır; JavaScript'te metotlar prototipte tek bir kez durur, tüm örnekler onu paylaşır.

```
1 // Bir nesnede bulunmayan özellik, prototip zincirinde yukarı doğru aranır.
2 // Object.create ile elle bir prototip bağı kuruyoruz (C++'ta olmayan,
3 // çalışma zamanına özgü bir mekanizma).
4 const katilimciProto = {
5   selamla() {
6     return `Merhaba, ben ${this.ad}`;
7   },
8 };
9
10 const katilimcil = Object.create(katilimciProto);
11 katilimcil.ad = "Ayşe";
12
13 console.log(katilimcil.selamla()); // "Merhaba, ben Ayşe"
14 // katilimcil'in kendi üzerinde "selamla" YOK; JS bunu prototipinde bulur.
15 console.log("kendi özelliği mi?", Object.hasOwn(katilimcil, "selamla")); // false
```

```
1 // Aynı davranış, ES6 class söz dizimiyle: bu, prototip mekanizmasının
2 // üstüne kurulmuş okunaklı bir kısayoldur ("syntactic sugar").
3 class Katilimci {
4   constructor(ad) {
5     this.ad = ad;
6   }
7
8   selamla() {
9     return `Merhaba, ben ${this.ad}`;
10  }
11 }
12
13 const katilimci2 = new Katilimci("Mehmet");
14 console.log(katilimci2.selamla()); // "Merhaba, ben Mehmet"
15 // class ile tanımlanan metotlar da aslında prototip üzerindedir.
16 console.log(
17   "selamla prototipte mi?",
18   Object.hasOwn(Katilimci.prototype, "selamla")
19 ); // true
```

Adım 4 (devamı)**>_ Terminal**`$ node prototip-sinif.js`

👁 Beklenen sonuç: `Object.hasOwn(katilimcil, "selamla")` false döner — selamla katilimcil'in kendi üzerinde değil, prototipindedir; aynı şekilde `class` ile tanımlanan `selamla` da `Katilimci.prototype` üzerinde durur.

🖥 Beklenen Çıktı

```
Merhaba, ben Ayşe
kendi özelliği mi? false
Merhaba, ben Mehmet
selamla prototipte mi? true
```

💡 İpucu

JavaScript motoru referansı kalmayan bir nesneyi otomatik temizler (çöp toplama/GC) — C++'taki `delete` ya da `RAII` (yıkıcı fonksiyon) ile bellek yönetiminin aksine, burada belleği elle serbest bırakmazsınız.

✅ Kontrol Noktası — devam etmeden doğrulayın

`deger-referans.js`, `closure.js` ve `prototip-sinif.js` dosyalarınızı `node` ile çalıştırıp yukarıdaki üç çıktıyı birebir gördüyseniz, JavaScript'in değer/ referans, closure ve prototip mekanizmalarını doğru kavramışsınız demektir — modüllere geçebilirsiniz.

Adım 5: ES modülleriyle bir fonksiyonu ayrı bir dosyaya taşıyın

❓ Ne yapacağız? `modul-disa-aktar.mjs` dosyanıza `export` ile bir `fiyatHesapla(birimFiyat, adet)` fonksiyonu yazacaksınız; `modul-ice-aktar.mjs` dosyanıza ise `import` ile bu fonksiyonu alıp 150 TL birim fiyat ve 3 adetle çağıracaksınız.

📖 Neden? `export/import` olmadan her `.js` dosyası aynı global kapsamı paylaşır — iki dosyada aynı isimli bir değişken tanımlarsanız biri diğerini ezer. ES modülü, her dosyaya C++'taki bir `.cpp/.h` çiftinin sağladığına benzer bir kapsam sınırı çizer: yalnızca `export` edilen isimler dışarıdan görünür.

```
1 // Adlandırılmış (named) export: bu dosyanın kapsamı korunur, yalnızca
2 // export edilen isimler dışarıdan içe aktarılabilir.
3 export function fiyatHesapla(birimFiyat, adet) {
4   return birimFiyat * adet;
5 }
```

```
1 // import ile önceki dosyadan adlandırılmış export'u alıp kullanma.
2 import { fiyatHesapla } from "../modul-disa-aktar.mjs";
3
4 const toplam = fiyatHesapla(150, 3);
5 console.log("toplam:", toplam); // 450
```

Adım 5 (devamı)**>_ Terminal**`$ node modul-ice-aktar.mjs`

👁 Beklenen sonuç: `modul-disa-aktar.mjs` dosyasını doğrudan çalıştırmanıza gerek yoktur — `modul-ice-aktar.mjs` onu `import` ile içe aktarıp kullanır.

🖥 Beklenen Çıktı`toplam: 450`**Ortak Hata / Tuzak**

Dosya uzantısını `.js` bırakırsanız Node bunu varsayılan CommonJS (`require`) modülü sanabilir ve `import/export` söz dizimine hata verir — bu yüzden bu iki dosyada uzantı bilinçli olarak `.mjs`'dir (ya da `package.json`'a `"type": "module"` eklenmelidir; Ünite 2'de Vite projenizde bunu göreceksiniz).

2 Modüller ve Araç Zinciri: ES Modülleri, npm, Vite

(bkz. *Ders notu Hafta 3, Ünite 2 — Modüller ve Araç Zinciri: ES Modülleri, npm, Vite.*)

Adım 6: Vite ile bir TypeScript projesi oluşturun

❓ Ne yapacağız? `web-lab` klasörünüzün yanında (aynı üst klasörde) `npm create vite@latest` komutuyla **saf TypeScript** (`vanilla-ts`) şablonlu yeni bir proje oluşturacak, projenizin adını istemci koyacak ve bağımlılıklarını indireceksiniz.

📌 Neden? Bu komut **React şablonunu değil**, saf TypeScript şablonunu seçmenizi ister — React Hf. 4'te aynı istemci klasörüne eklenecektir, bugün henüz yoktur. Vite, kaynak dosyalarınızı (TS/CSS/HTML) tarayıcıya tek tek göndermek yerine derleyip paketleyen (bundle) bir araçtır; npm de bu aracın (ve TypeScript derleyicisinin) kendisini indiren paket yöneticisidir.

- `npm create vite@latest`
- İstendiğinde proje adı olarak `istemci` yazın.
- Şablon listesinde `TypeScript` → `vanilla-ts` seçin (**React değil**).

>_ Terminal

```
$ cd istemci
$ npm install
```

👁 Beklenen sonuç: `istemci` klasöründe `index.html`, `src/main.ts`, `tsconfig.json`, `package.json` dosyaları ve `npm install` sonrası bir `node_modules` klasörü görünür.

🖥 Beklenen Çıktı`added 20 packages in 2s`

Adım 7: package.json ve tsconfig.json dosyalarını inceleyin

? Ne yapacağız? Oluşan package.json dosyanızı açıp scripts alanındaki dev/build/preview komutlarını, devDependencies alanındaki typescript/vite/@types/node sürümlerini inceleyeceksiniz; referans dosyadaki engines alanını kendi dosyanıza da ekleyeceksiniz.

i Neden? package.json projenizin “alışveriş listesi”dir: hangi paketin (bağımlılığın) hangi sürümde gerektiğini tutar. dependencies çalışma zamanında gereken paketleri, devDependencies ise yalnızca geliştirme/derleme sırasında gereken araçları (TypeScript, Vite) listeler — ikisi karıştırılmaz. engines alanı ise projenizin hangi Node sürümüyle test edildiğini belgeler; bu hafta Node 24 LTS sabitlenmiştir.

```

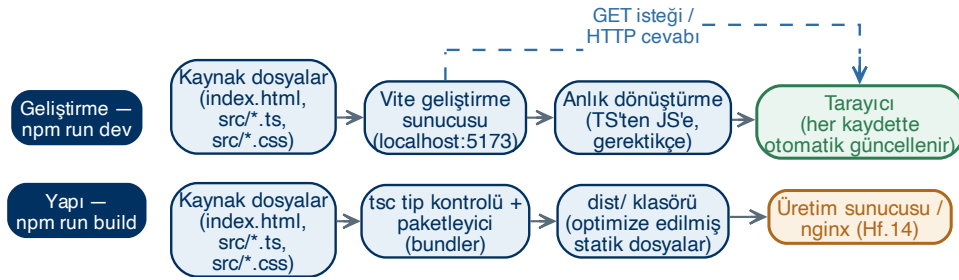
1 {
2   "name": "istemci",
3   "private": true,
4   "version": "0.0.0",
5   "type": "module",
6   "engines": {
7     "node": "^24.13.0 || >=24.13.0"
8   },
9   "scripts": {
10    "dev": "vite",
11    "build": "tsc && vite build",
12    "preview": "vite preview"
13  },
14  "devDependencies": {
15    "typescript": "^7.0.2",
16    "vite": "^8.2.2",
17    "@types/node": "^24.13.3"
18  }
19 }
```

👁 Beklenen sonuç: Referans dosyadaki "typescript": "^7.0.2", "vite": "^8.2.2" sürümleriyle kendi package.json'ınızdaki (npm create vite@latest'in o an çektiği) sürümler birebir aynı olmayabilir — bu normaldir; önemli olan scripts alanının aynı üç komutu (dev, build, preview) içermesidir.

Adım 8: Geliştirme sunucusunu başlatın, dev ile build farkını gözlemleyin

? Ne yapacağız? npm run dev komutuyla geliştirme sunucusunu başlatıp tarayıcıda http://localhost:5173 adresini açacaksınız; src/main.ts dosyasında küçük bir metin değişikliği yapıp kaydettiğinizde tarayıcının otomatik güncellendiğini göreceksiniz. Ardından sunucuyu (Ctrl+C) durdurup npm run build ile üretim çıktısını oluşturacaksınız.

i Neden? Vite'in geliştirme sunucusu da bir HTTP sunucusudur — localhost:5173'te dinler, tarayıcı ona bir GET isteği gönderir, o da index.html ve modülleri cevap olarak döner (Hf. 1 Ünite 2'nin “web sunucusu” kavramı burada ilk kez somut bir araçla karşınıza çıkıyor). dev kodu tam derlemeden hızlı geri bildirim verir ve yalnızca geliştirme sırasında kullanılır; build kodu tsc ile tam tip kontrolünden geçirip optimize eder, gerçek bir sunucuya yüklenebilecek statik bir dist/ klasörü üretir — geliştirme sunucusu üretimde asla kullanılmaz.

Adım 8 (devamı)

```
>_ Terminal
$ npm run dev
```

👁 **Beklenen sonuç:** Tarayıcıda Vite’in varsayılan karşılama sayfası (dönen Vite logosu, “Hello, Vite!” benzeri bir başlık) görünür; `src/main.ts`’te yaptığınız değişiklik sayfayı yenilemenize gerek kalmadan anında yansır.

```
>_ Terminal
$ npm run build
```

👁 **Beklenen sonuç:** Komut hatasız bitir ve istemci klasöründe yeni bir `dist/` klasörü oluşur — içinde `index.html` ve `assets/` altında paketlenmiş CSS/JS dosyaları bulunur.

Beklenen Çıktı

```
vite v8.2.2 building client environment for production...
transforming...
dist/index.html          1.76 kB
dist/assets/index-CfPMV_iP.css 2.29 kB
dist/assets/index-BWubBFDC.js 3.30 kB

built in 362ms
```

✓ Kontrol Noktası — devam etmeden doğrulayın

`npm run dev` ile tarayıcıda Vite’in karşılama sayfasını gördüyseniz ve `npm run build` bir `dist/` klasörü ürettiyse, araç zincirinizi doğru kurmuşsunuz demektir — artık bu projeyi Hafta 2’nin sayfasına dönüştürebilirsiniz.

3 TypeScript Tip Sistemi: Etkinlik, Koltuk, Bilet

(bkz. Ders notu Hafta 3, Ünite 3 — TypeScript Tip Sistemi: Etkinlik, Koltuk, Bilet.)

Adım 9: Etkinlik arayüzünü tanımlayın

🔍 **Ne yapacağız?** istemci/src klasörünüzde `tipler.ts` adlı yeni bir dosya oluşturup içine Etkinlik arayüzünü (`id`, `baslik`, `tarih`, `mekan`, `kapasite`, `biletFiyati` alanlarıyla) yazacaksınız.

Adım 9 (devamı)

i Neden? Bir interface, bir nesnenin “şeklini” — hangi alanların hangi tipte olması gerektiğini — önceden tanımlar; tıpkı bir başvuru formunun hangi alanın metin, hangisinin sayı olacağını önceden belirlemesi gibi. `biletFiyati` alanı bilinçli bir ektir — Ünite 4'te koltuk seçiminde “toplam tutar” hesaplarırken kullanacaksınız; ad ve tip bu haftadan itibaren sabittir, sonraki haftalar değiştirmez.

```
1 export interface Etkinlik {
2   id: number;
3   baslik: string;
4   tarih: string; // ISO 8601 tarih dizesi, ör. "2026-11-14"
5   mekan: string;
6   kapasite: number;
7   biletFiyati: number; // TL cinsinden birim koltuk fiyatı
8 }
```

👁 Beklenen sonuç: Dosyayı kaydettiğinizde editörünüzde (VS Code) hiçbir kırmızı çizgi görünmez — arayüz tanımı tek başına bir hata üretmez, yalnızca bir “şablon” oluşturur.

Adım 10: Koltuk ve Bilet tiplerini union ve readonly ile, `Liste<T>`'yi generic ile tanımlayın

? Ne yapacağız? Aynı tipler `.ts` dosyanıza sırasıyla `KoltukDurumu` union tipini (`'bos' | 'secili' | 'dolu'`) ve `readonly` alanlar içeren `Koltuk` arayüzünü, ardından `BiletDurumu` union tipini ve `Bilet` arayüzünü, son olarak YG I'nin `Liste<T>` şablon sınıfına karşılık gelen generic `Liste<T>` arayüzünü ekleyeceksiniz.

i Neden? Union tipi (`'bos' | 'secili' | 'dolu'`) bir değişkenin yalnızca önceden izin verilen değerlerden birini almasını sağlar — dördüncü bir değer (`'rezerve'` gibi) tsc'de hata verir. `readonly`, `id` gibi kimlik alanlarının ilk atamadan sonra değişmemesini garanti eder. `Liste<T>` ise YG I'nin kendi `Liste<T>` şablon sınıfıyla aynı fikri taşır: rafın (arayüzün) kendisi tek bir tasarımdır, hangi tip için çalışacağı (`Liste<Etkinlik>`, `Liste<Koltuk>`) kullanım anında belirlenir.

```
1 export type KoltukDurumu = 'bos' | 'secili' | 'dolu';
2
3 export interface Koltuk {
4   readonly id: number;
5   readonly etkinlikId: number;
6   sıra: number;
7   koltukNo: number;
8   durum: KoltukDurumu;
9 }
```

```
1 export type BiletDurumu = 'olusturuldu' | 'iptal';
2
3 export interface Bilet {
4   readonly id: number;
5   readonly koltukId: number;
6   kullanıcıAdi: string; // Hf.10'a kadar gerçek oturum yok, düz metin
7   satınAlmaZamani: string; // ISO 8601 zaman damgası
8   durum: BiletDurumu;
9 }
```

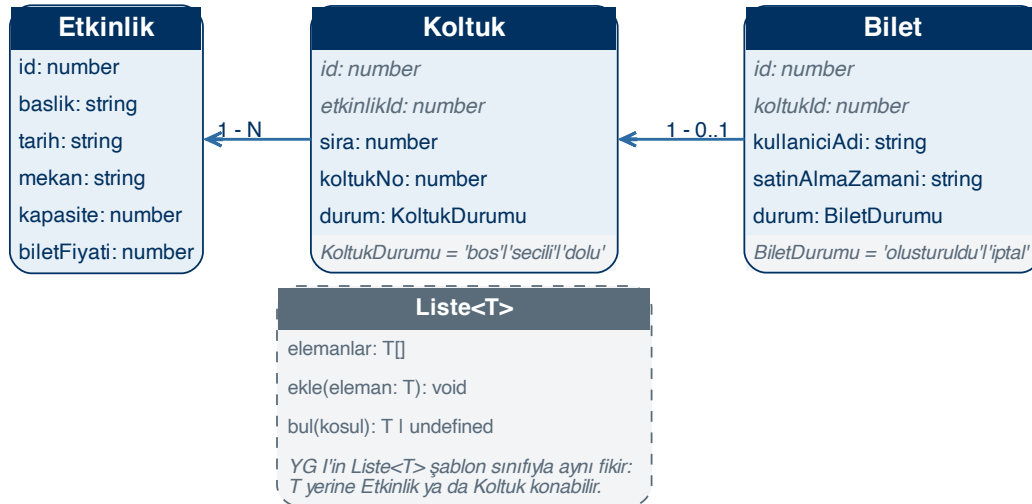
```
1 // YG I'nin kendi Liste<T> şablon sınıfıyla doğrudan kıyas: bu arayüz de
2 // hangi tip için çalışacağını KULLANIM ANINDA (Liste<Etkinlik>
```

Adım 10 (devamı)

```

3 // Liste<Koltuk> gibi) belirleyen bir kalıptır.
4 export interface Liste<T> {
5   elemanlar: T[];
6   ekle(eleman: T): void;
7   bul(id: number): T | undefined;
8 }

```



👁 **Beklenen sonuç:** Dört tip/arayüz de (Etkinlik, Koltuk, Bilet, Liste<T>) hatasız derlenir; VS Code'da Koltuk yazıp nokta (.) tuşuna bastığınızda editör size id, etkinlikId, sıra, koltukNo, durum alanlarını otomatik tamamlama listesinde önerir.

Adım 11: Bilerek hatalı bir atama yapıp tsc --noEmit hatasını gözlemleyin

❓ **Ne yapacağız?** tipler.ts'in altına, yalnızca bu deney için, geçici bir test satırı ekleyeceksiniz: bir Koltuk değişkeni oluşturup durum alanına 'rezerve' atayacaksınız. Ardından terminalde istemci klasöründe npx tsc --noEmit komutunu çalıştırıp editördeki kırmızı çizgiyle aynı hatayı terminalde de göreceksiniz. Deneyi bitirince bu geçici satırı silin.

📌 **Neden?** tsc --noEmit kodu JavaScript'e çevirmeden yalnızca tip hatalarını denetler — editördeki kırmızı çizgilerin build/CI öncesi toplu bir doğrulamasıdır. JavaScript'te bu hata sessizce geçirdi; kod çalışırken, belki bir kullanıcının önünde patlardı. TypeScript bu hatayı siz kodu yazarken yakalar.

```

>_ Terminal
$ npx tsc --noEmit

```

👁 **Beklenen sonuç:** Terminalde Type '"rezerve"' is not assignable to type 'KoltukDurumu' benzeri bir hata satırı görünür — çünkü 'rezerve', KoltukDurumu union'ında tanımlı üç değerden biri değildir. Geçici satırı sildikten sonra aynı komutu tekrar çalıştırdığınızda **hiçbir çıktı vermeden** (hatasız) biter.

Adım 11 (devamı)**Ortak Hata / Tuzak**

tsc --noEmit hatasız bittiğinde terminalde hiçbir satır görünmez — bu bir arıza değil, “her şey yolunda” anlamına gelir. Çoğu öğrenci boş çıktıyı “komut çalışmadı” sanıp tekrar tekrar çalıştırır; komutun çıkış koduna (echo \$? ile 0 mı) bakmak daha güvenilirdir.

Adım 12: Sabit Etkinlik[] ve Koltuk[] dizilerini yazın

❓ Ne yapacağız? src klasörünüzde veri.ts adlı yeni bir dosya oluşturup içine tipler.ts'ten Etkinlik ve Koltuk tiplerini içe aktaracak, ardından en az 3 elemanlı sabit bir etkinlikler: Etkinlik[] dizisi ve Bahar Konseri'ne ait 3 sıra × 4 koltukluk sabit bir koltuklar: Koltuk[] dizisi (iki koltuğu 'dolmuş' olarak) yazacaksınız.

❗ Neden? Bu hafta sunucu (FastAPI) henüz yok — veri, tıpkı Hf. 2'de HTML'e elle yazdığınız kartlar gibi, şimdilik TypeScript dosyasına gömülü sabit bir dizidir. Bu dizi, Ünite 4'te DOM ile dinamik olarak kartlara/koltuklara dönüştürülecek tek kaynak olacaktır.

```
1 export const etkinlikler: Etkinlik[] = [
2   {
3     id: 1,
4     baslik: 'Bahar Konseri',
5     tarih: '2026-05-15',
6     mekan: 'Açık Hava Amfisi',
7     kapasite: 500,
8     biletFiyati: 150,
9   },
10  {
11    id: 2,
12    baslik: 'Kariyer Günleri',
13    tarih: '2026-05-22',
14    mekan: 'Kongre Merkezi',
15    kapasite: 800,
16    biletFiyati: 0,
17  },
18  {
19    id: 3,
20    baslik: 'Tiyatro Gecesi',
21    tarih: '2026-05-29',
22    mekan: 'Kültür Merkezi',
23    kapasite: 250,
24    biletFiyati: 200,
25  },
26 ];

1 // Bahar Konseri (id: 1) için 3 sıra x 4 koltuklu örnek ızgara; B2 ve C4
2 // önceden "dolmuş" olarak işaretlenmiştir (tıklanamaz).
3 export const koltuklar: Koltuk[] = [
4   { id: 1, etkinlikId: 1, sıra: 1, koltukNo: 1, durum: 'bos' },
5   { id: 2, etkinlikId: 1, sıra: 1, koltukNo: 2, durum: 'bos' },
6   { id: 3, etkinlikId: 1, sıra: 1, koltukNo: 3, durum: 'bos' },
7   { id: 4, etkinlikId: 1, sıra: 1, koltukNo: 4, durum: 'bos' },
8   { id: 5, etkinlikId: 1, sıra: 2, koltukNo: 1, durum: 'bos' },
9   { id: 6, etkinlikId: 1, sıra: 2, koltukNo: 2, durum: 'dolmuş' },
10  { id: 7, etkinlikId: 1, sıra: 2, koltukNo: 3, durum: 'bos' },
```

Adım 12 (devamı)

```

11 { id: 8, etkinlikId: 1, sıra: 2, koltukNo: 4, durum: 'bos' },
12 { id: 9, etkinlikId: 1, sıra: 3, koltukNo: 1, durum: 'bos' },
13 { id: 10, etkinlikId: 1, sıra: 3, koltukNo: 2, durum: 'bos' },
14 { id: 11, etkinlikId: 1, sıra: 3, koltukNo: 3, durum: 'bos' },
15 { id: 12, etkinlikId: 1, sıra: 3, koltukNo: 4, durum: 'dolu' },
16 ];

```

Beklenen sonuç: `npx tsc --noEmit` yine hatasız biter — dizilerdeki her nesnenin alan adları ve tipleri Etkinlik/Koltuk arayüzleriyle birebir eşleşir.

✓ Kontrol Noktası — devam etmeden doğrulayın

`tipler.ts` dosyanızda dört tip/arayüzü tanımladıysanız, 'rezerve' deneyinde `tsc` hatasını gördüyseniz ve `veri.ts`'teki iki dizi hatasız derleniyorsa, Etkinlik/Bilet Sisteminin alan modeli tipleri hazır demektir — artık bu veriyi sayfada canlandırabilirsiniz.

4 Asenkron JavaScript ve DOM: Olay Döngüsü, Promise, async/await

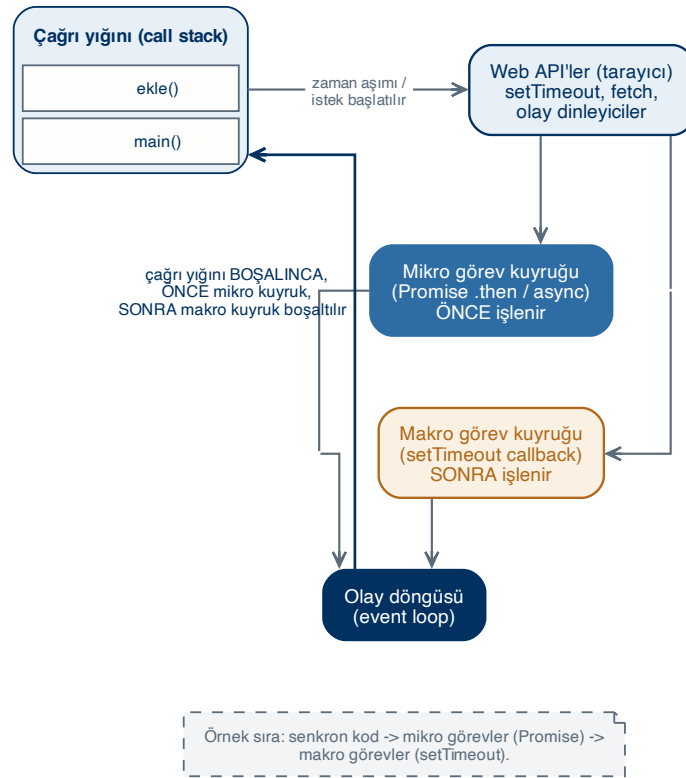
(bkz. Ders notu Hafta 3, Ünite 4 — Asenkron JavaScript ve DOM: Olay Döngüsü, Promise, async/await.)

Adım 13: Olay döngüsü sırasını tahmin edip Promise ile karşılaştırın

? Ne yapacağız? `js-lab` klasörünüze `olay-dongusu.js` ve `promise-async.js` dosyalarını ekleyin. Önce aşağıdaki `olay-dongusu.js` kodunu okuyup dört `console.log`'un hangi sırayla (A, B, C, D) yazdıracağını tahmin edin, sonra kodu yazıp `node` ile çalıştırarak tahmininizi doğrulayın. Aynı şeyi `promise-async.js` için de — bu kez `.then()` zinciri ile `async/await` yazımını karşılaştırarak — tekrarlayın.

i Neden? JavaScript **tek** iş parçacığında çalışır — YG II'de gördüğünüz `std::thread` gerçek paralel iş parçacıkları açıp birden fazla kodun gerçekten aynı anda çalışmasını sağlar; JS'te "eşzamanlılık" gerçek paralellik değildir, bekleyen işlerin sıraya girip tek tek işlenmesiyle sağlanır. Çağrı yığını tamamen boşalmadan görev kuyruğundaki hiçbir iş alınmaz; mikro görev kuyruğu (Promise) her zaman makro görev kuyruğundan (`setTimeout`) önce boşaltılır — `setTimeout(fn, 0)` bile "hemen" değil "sıra gelince" demektir.

Adım 13 (devamı)



```

1 // JS tek iş parçacığında çalışır: çağrı yığını tamamen boşalmadan görev
2 // kuyruğundaki hiçbir iş alınmaz. setTimeout(fn, 0) bile "hemen" değil,
3 // "sıra gelince" demektir; mikro görev kuyruğu (Promise) makro görev
4 // kuyruğundan (setTimeout) ÖNCE boşaltılır.
5 console.log("A"); // senkron
6 setTimeout(() => console.log("B"), 0); // makro görev kuyruğu
7 Promise.resolve().then(() => console.log("C")); // mikro görev kuyruğu
8 console.log("D"); // senkron
9
10 // Beklenen sıra: A, D, C, B

```

> Terminal

```
$ node olay-dongusu.js
```

👁 **Beklenen sonuç:** Sıra A, D, C, B'dir — senkron kod (A, D) önce tamamen biter, sonra mikro görev kuyruğundaki Promise (C), en son makro görev kuyruğundaki setTimeout (B) çalışır.

🖥 Beklenen Çıktı

```

A
D
C
B

```

```

1 // Bir Promise: "şu an sonucu yok ama ileride ya başarıyla ya hatayla
2 // sonuçlanacak" bir söz. Burada gecikmeli bir "bilet oluşturma" işlemini

```

Adım 13 (devamı)

```

3 // setTimeout ile sarıp .then().catch() zinciriyle tüketiyoruz.
4 function biletOlusturSoz(ad) {
5   return new Promise((resolve) => {
6     setTimeout(() => {
7       resolve(`${ad} için bilet oluşturuldu`);
8     }, 200);
9   });
10 }
11
12 console.log("[then] istek gönderildi");
13 biletOlusturSoz("Ayşe")
14   .then((sonuc) => {
15     console.log("[then] sonuç:", sonuc);
16   })
17   .catch((hata) => {
18     console.error("[then] hata:", hata);
19   });

```

```

1 // Aynı işlem, async/await ile senkron görünümlü biçimde: await yalnızca
2 // içinde bulunduğu async fonksiyonu duraklatır, TÜM programı bloklamaz.
3 async function biletOlusturAsync(ad) {
4   console.log("[async] istek gönderildi");
5   const sonuc = await biletOlusturSoz(ad);
6   console.log("[async] sonuç:", sonuc);
7   return sonuc;
8 }
9
10 biletOlusturAsync("Mehmet");

```

>_ Terminal

\$ node promise-async.js

👁 Beklenen sonuç: İlk iki satır ([then]/[async] “istek gönderildi”) hemen yazılır; await’in yalnızca içinde bulunduğu biletOlusturAsync fonksiyonunu duraklattığını, tüm programı bloklamadığını buradan görürsünüz. Son iki satır yaklaşık 200 ms sonra — kayıt sırasına göre önce then zinciri, sonra async/await — yazılır.

🖥 Beklenen Çıktı

```

[then] istek gönderildi
[async] istek gönderildi
[then] sonuç: Ayşe için bilet oluşturuldu
[async] sonuç: Mehmet için bilet oluşturuldu

```

Adım 14: index.html’i Vite girişine uyarlayıp koltuk durum stillerini ekleyin

❓ Ne yapacağız? istemci/index.html dosyasındaki Vite’in varsayılan içeriğini silip, Hf.2’nin header/nav/main/footer iskeletini — üstte etkinlik listesi <section>’ı, altta koltuk planı ve bilet formu <section>’ı olacak biçimde — yazacak, en sonda src/main.ts’e bağlanan <script type="module"> etiketini ekleyeceksiniz. istemci/src/style.css dosyanıza da Hf. 2’nin stiller.css’ini uyarlayıp .koltuk.secili ve .koltuk.dolu durum stillerini ekleyeceksiniz.

Adım 14 (devamı)

i Neden? Hf. 2'nin iki ayrı statik dosyası (index.html, etkinlik-detay.html) bu hafta tek bir Vite girişinde birleşir — bu, Hf. 1 Ünite 3'ün SPA (*Single Page Application*) fikrinin ilk somut örneğidir: tek sayfa, birden çok bölüm. Semantik iskelet değişmez, yalnızca üstüne davranış eklenir. .koltuk.secili/.koltuk.dolu sınıfları, birazdan yazacağınız TypeScript'in classList ile açıp kapatacağı görsel durumlardır.

```

1 <!DOCTYPE html>
2 <html lang="tr">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1">
6   <title>KTÜ Etkinlik ve Bilet Sistemi</title>
7 </head>
8 <body>
9 <header>
10  <nav class="site-nav">
11    <ul>
12      <li><a href="#etkinlikler">Etkinlikler</a></li>
13      <li><a href="#bilet-al">Bilet Al</a></li>
14    </ul>
15  </nav>
16 </header>
17
18 <main>

```

```

1 <script type="module" src="/src/main.ts"></script>

```

```

1 .koltuk.secili {
2   background-color: var(--renk-yesil);
3   color: #FFFFFF;
4   border-color: var(--renk-yesil);
5 }
6
7 .koltuk.dolu {
8   background-color: var(--renk-acikgri);
9   border-color: var(--renk-gri);
10  color: var(--renk-gri);
11  cursor: not-allowed;
12 }

```

👁 Beklenen sonuç: npm run dev çalışırken tarayıcıyı yenilediğinizde Hf. 2'deki gibi lacivert bir üst bilgi, gezinme menüsü ve "Bilet Al" başlığı görünür; etkinlik listesi ve koltuk planı alanları henüz boştur — içeriklerini sonraki adımlarda TypeScript ile üreteceksiniz.

Adım 15: Etkinlik kartlarını veri.ts'ten DOM ile üretin

? Ne yapacağız? src klasörünüzde etkinlikListesi.ts adlı yeni bir dosya oluşturup içine, bir kapsayıcı öge ve bir Etkinlik[] dizisi alan, her etkinlik için document.createElement ile bir article.etkinlik-karti oluşturup kapsayıcıya ekleyen etkinlikListesiUret fonksiyonunu yazacaksınız.

i Neden? Hf. 2'de her kartı elle HTML'e yazmıştınız; artık kartlar veri.ts'teki etkinlikler dizisinden — Ünite 1'de öğrendiğiniz forEach ile — üretilir. Dizi değişirse (yani bir etkinlik eklenirse) HTML'e dokunmanıza gerek kalmaz; DOM API'si (querySelector ile kapsayıcıyı bulma, createElement/appendChild ile öge ekleme) bu üretimi sizin yerini-

Adım 15 (devamı)

ze yapar.

```

1 // etkinlikler dizisinden DOM ile kart listesi üretir; HTML'e elle
2 // kart yazmak yerine, her etkinlik için createElement ile bir
3 // article.etkinlik-karti oluşturup kapsayıcıya ekler (forEach).
4 export function etkinlikListesiUret(
5   kapsayici: HTMLElement,
6   etkinlikler: Etkinlik[]
7 ): void {
8   kapsayici.innerHTML = '';
9
10  etkinlikler.forEach((etkinlik) => {
11    const kart = document.createElement('article');
12    kart.className = 'etkinlik-karti';
13
14    const baslik = document.createElement('h3');
15    baslik.textContent = etkinlik.baslik;
16
17    const bilgi = document.createElement('p');
18    bilgi.textContent = `${etkinlik.tarih} · ${etkinlik.mekan}`;
19
20    const kapasite = document.createElement('p');
21    kapasite.textContent = `Kapasite: ${etkinlik.kapasite} · Bilet: ${etkinlik.
      biletFiyati} TL`;
22
23    kart.append(baslik, bilgi, kapasite);
24    kapsayici.appendChild(kart);
25  });
26 }

```

👁 Beklenen sonuç: Bu fonksiyonu henüz çağırmadınız — bir sonraki adımda main.ts'te çağırduğunuzda tarayıcıda 3 etkinlik kartı (Bahar Konseri, Kariyer Günleri, Tiyatro Gecesi) görünecektir.

Adım 16: Koltuk ızgarasını üretip her koltuğa tıklama davranışı ekleyin

❓ Ne yapacağız? src klasörünüzde koltukPlani.ts adlı yeni bir dosya oluşturup içine, koltuklar dizisinden koltuk ızgarasını DOM ile üreten koltukPlaniUret fonksiyonunu ve her koltuğa bir click dinleyicisi ekleyen koltukTiklamaIsleyici yardımcı fonksiyonunu yazacaksınız.

📌 Neden? Her koltuk ögesine addEventListener('click', ...) ile bir olay dinleyicisi eklersiniz — kullanıcı koltuğa tıkladığında tarayıcı bu dinleyiciyi çağırır (Hf. 1 Ünite 4'ün “kapıyı çaldığında ne olacağını önceden tanımlama” fikri). durum === 'dolü' kontrolü bir tip daraltma örneğidir: KoltukDurumu union'ı üç değerden birine sahipken, bu kontrolden sonra TypeScript durum'un kalan iki değerden biri olduğunu “bilir”.

```

1 // Bir koltuğa tıklandığında durum geçişini uygular: bos -> secili ya da
2 // secili -> bos. dolu koltuklar tıklanamaz (hiçbir şey yapılmaz).
3 function koltukTiklamaIsleyici(
4   koltuk: Koltuk,
5   eleman: HTMLDivElement,
6   degisiklikOldu: () => void
7 ): void {
8   if (koltuk.durum === 'dolü') {
9     return;

```

Adım 16 (devamı)

```

10 }
11
12 koltuk.durum = koltuk.durum === 'bos' ? 'secili' : 'bos';
13 eleman.classList.toggle('secili', koltuk.durum === 'secili');
14 degisiklikOldu();
15 }

1 // koltuklar dizisinden koltuk ızgarasını DOM ile üretir; her koltuğa
2 // bir click dinleyicisi ekler.
3 export function koltukPlaniUret(
4   kapsayici: HTMLElement,
5   koltuklar: Koltuk[],
6   degisiklikOldu: () => void
7 ): void {
8   kapsayici.innerHTML = '';
9
10  koltuklar.forEach((koltuk) => {
11    const eleman = document.createElement('div');
12    eleman.className = 'koltuk';
13    eleman.textContent = koltukEtiketi(koltuk);
14
15    if (koltuk.durum === 'dolu') {
16      eleman.classList.add('dolu');
17    }
18
19    eleman.addEventListener('click', () => {
20      koltukTiklamaIsleyici(koltuk, eleman, degisiklikOldu);
21    });
22
23    kapsayici.appendChild(eleman);
24  });
25 }

```

👁 **Beklenen sonuç:** Bu iki fonksiyonu da henüz çağırmadınız — `main.ts`'e bağladığınızda 'dolu' koltuklar (B2, C4) açık gri ve tıklanamaz, diğerleri açık mavi ve tıklanabilir görünecektir.

✅ **Kontrol Noktası — devam etmeden doğrulayın**

`index.html/style.css` uyarlamanızı ve `etkinlikListesi.ts/koltukPlani.ts` dosyalarınızı yazdıysanız (henüz `main.ts`'e bağlamadan), `npx tsc --noEmit`'in hatasız bittiğini doğrulayın — birazdan bu fonksiyonları sayfaya bağlayacaksınız.

Adım 17: Seçim özetini ve async bilet gönderimini yazın

❓ **Ne yapacağız?** `src` klasörünüzde `secimOzeti.ts` adlı dosyayı oluşturup seçili koltuk sayısını ve toplam tutarı (`biletFiyati × sayı`) hesaplayıp ilgili paragrafı güncelleyen `secimOzetiGuncelle` fonksiyonunu yazacaksınız. Ardından `biletForm.ts` dosyasını oluşturup formun submit olayında `preventDefault()` çağırarak, seçim özetini konsola basan ve `setTimeout+Promise` ile "bilet oluşturuluyor" gecikmesini simüle eden `async biletOlustur()` fonksiyonunu çağırarak `biletFormBaglan` fonksiyonunu yazacaksınız.

📌 **Neden?** `action` özniteliği olmadan bir form gönderildiğinde tarayıcı varsayılan olarak sayfayı yeniden yükler — Hf. 2'de gördüğümüz "hiçbir şey olmuyor" görünümünün asıl nedeni budur. `event.preventDefault()` bu varsayılan davranışı iptal eder; artık JavaScript

Adım 17 (devamı)

sayfanın yenilenmesini engelleyip kendi mantığınızı (seçim özetini konsola basmak, sunucuya gönderiyormuş gibi davranmak) çalıştırabilir. `async/await` burada bir sunucu isteğinin (Hf. 6'da gerçek olacak) yer tutucusudur — `setTimeout` ile sarılmış bir Promise, gerçek bir ağ gecikmesini simüle eder.

```

1 // Seçili koltuk sayısını ve toplam tutarı (biletFiyati x sayı) hesaplayıp
2 // ilgili DOM öğesini günceller.
3 export function secimOzetiGuncelle(
4   eleman: HTMLElement,
5   koltuklar: Koltuk[],
6   biletFiyati: number
7 ): number {
8   const seciliSayisi = koltuklar.filter((k) => k.durum === 'secili').length;
9   const toplamTutar = seciliSayisi * biletFiyati;
10
11   eleman.textContent = `Seçili koltuk: ${seciliSayisi}, Toplam: ${toplamTutar} TL`;
12
13   return seciliSayisi;
14 }

1 // setTimeout ile sarılmış bir Promise, "bilet oluşturuluyor" gecikmesini
2 // simüle eder; async/await ile senkron görünümlü biçimde tüketilir.
3 async function biletOlustur(ozet: SecimOzeti): Promise<Bilet> {
4   return new Promise((resolve) => {
5     setTimeout(() => {
6       resolve({
7         id: Date.now(),
8         koltukId: ozet.koltukIdleri[0] ?? 0,
9         kullanıcıAdi: ozet.kullanıcıAdi,
10        satınAlmaZamani: new Date().toISOString(),
11        durum: 'olusturuldu',
12      });
13    }, 1000);
14  });
15 }

1 // Formun submit olayında preventDefault() çağrılır (sayfa yenilenmez);
2 // seçim özeti konsola basılır, ardından async biletOlustur() çağrılır
3 // sonucu bekleyen bir onay logu yazılır.
4 export function biletFormBaglan(
5   form: HTMLFormElement,
6   adSoyadGirdi: HTMLInputElement,
7   koltuklar: Koltuk[]
8 ): void {
9   form.addEventListener('submit', (olay) => {
10     olay.preventDefault();
11
12     const seciliKoltuklar = koltuklar.filter((k) => k.durum === 'secili');
13     const ozet: SecimOzeti = {
14       kullanıcıAdi: adSoyadGirdi.value,
15       koltukIdleri: seciliKoltuklar.map((k) => k.id),
16     };
17
18     console.log('Bilet özeti:', ozet);
19
20     biletOlustur(ozet).then((bilet) => {
21       console.log('Bilet oluşturuldu:', bilet);

```

Adım 17 (devamı)

```

22 | });
23 | });
24 | }

```

👁 **Beklenen sonuç:** Bu fonksiyonları da henüz çağırmadınız; bir sonraki adımda hepsini `main.ts`'e bağlayınca sayfa tam olarak canlanacaktır.

Adım 18: `main.ts` ile tüm modülleri bağlayıp tarayıcıda test edin

❓ **Ne yapacağız?** `src/main.ts` dosyasının içeriğini silip, `querySelector` ile beş DOM ögesi (etkinlik listesi kapsayıcısı, koltuk planı kapsayıcısı, seçim özeti paragrafı, bilet formu, ad-soyad girdisi) seçen, ardından `etkinlikListesiUret/koltukPlaniUret/ biletFormBaglan` fonksiyonlarını çağırıp sayfayı ilk kez oluşturan giriş noktası kodunu yazacaksınız.

📌 **Neden?** Her modül (`etkinlikListesi.ts`, `koltukPlani.ts`, `secimOzeti.ts`, `biletForm.ts`) kendi işini yapan, birbirinden bağımsız bir fonksiyon dışa aktarır — Ünite 1'in ES modülleri fikri burada somutlaşır. `main.ts` bu parçaları bir araya getiren tek giriş noktasıdır; her koltuk tıklandığında `degisiklikOldu` geri çağırımı (*callback*) üzerinden `secimOzetiGuncelle`'yi tetikler.

```

1 // Tüm modülleri içe aktarıp sayfayı ilk kez oluşturan giriş noktası.
2 const etkinlikListesiElemani = document.querySelector<HTMLDivElement>('#etkinlik-
   listesi')!;
3 const koltukPlaniElemani = document.querySelector<HTMLDivElement>('#koltuk-plani')!;
4 const secimOzetiElemani = document.querySelector<HTMLParagraphElement>('#secim-ozeti'
   )!;
5 const biletFormuElemani = document.querySelector<HTMLFormElement>('#bilet-formu')!;
6 const adSoyadElemani = document.querySelector<HTMLInputElement>('#ad-soyad')!;
7
8 // Koltuk planı, index 0'daki etkinliğin biletFiyati'nı kullanır (Bahar
9 // Konseri) – koltuklar dizisindeki tüm koltuklar bu etkinliğe aittir.
10 const seciliEtkinlik = etkinlikler.find((e) => e.id === koltuklar[0].etkinlikId!);
11
12 function ozetiYenile(): void {
13   secimOzetiGuncelle(secimOzetiElemani, koltuklar, seciliEtkinlik.biletFiyati);
14 }
15
16 etkinlikListesiUret(etkinlikListesiElemani, etkinlikler);
17 koltukPlaniUret(koltukPlaniElemani, koltuklar, ozetiYenile);
18 biletFormBaglan(biletFormuElemani, adSoyadElemani, koltuklar);
19
20 ozetiYenile();

```

```

>_ Terminal
$ npm run dev

```

👁 **Beklenen sonuç:** Tarayıcıda `http://localhost:5173` adresinde: 3 etkinlik kartı görünür; boş bir koltuğa tıklandığında rengi “seçili” durumuna döner ve “Seçili koltuk: 1, Toplam: 150 TL” metni anlık günceller; aynı koltuğa tekrar tıklayınca boşa döner; “dolmuş” koltuklara tıklamanın hiçbir etkisi olmaz. Bilet formunda ad-soyad yazıp “Gönder”e bastığınızda sayfa yenilenmez, tarayıcı konsoluna (DevTools → Console) önce seçim özeti, yaklaşık 1 saniye sonra da “Bilet oluşturuldu” onay logu yazılır.

Adım 18 (devamı)**Beklenen Çıktı**

```
Bilet özeti: { kullanıcıAdi: 'Ayşe', koltukIdleri: [ 1 ] }
Bilet oluşturuldu: { id: 1799..., koltukId: 1, kullanıcıAdi: 'Ayşe',
  satınAlmaZamani: '2026-...', durum: 'olusturuldu' }
```

Adım 19: Kendiniz ekleyin: koltuk ızgarasına yeni bir sıra ekleyin

? Ne yapacağız? veri.ts'teki koltuklar dizisine, 4. sıra için dört yeni Koltuk nesnesi (sıra: 4, koltukNo: 1..4, hepsi durum: 'bos', id değerleri 13–16) elle ekleyin. Ardından style.css'teki .koltuk-planı kuralındaki grid-template-rows değerini repeat(3, 1fr)'den repeat(4, 1fr)'e güncelleyin.

i Neden? Bu görev, Ünite 3'ün tip sisteminin size sağladığı güvenceyi elle deneyimlemenizi sağlar: yeni nesnelerden birinde bir alanı unutursanız (ör. durum eksikse) tsc sizi hemen uyarır — kodu çalıştırmadan, editördeyen.

👁 Beklenen sonuç: Tarayıcıyı yenilediğinizde koltuk planı artık 4 sıra × 4 sütun (16 koltuk) görünür; yeni eklediğiniz 4. sıradaki koltuklar 'bos' olduğundan tıklanabilir olmalıdır.

💡 İpucu

Yeni id değerlerinin mevcut 12 koltuktan (1–12) farklı olduğundan emin olun — aynı id'yi tekrar kullanmak, iki farklı koltuğun aynı kimliği paylaşmasına, dolayısıyla seçim mantığının karışmasına yol açar.

✅ Kontrol Noktası — devam etmeden doğrulayın

Etkinlik listesinin 3 kart, koltuk planının artık 4×4 ızgara gösterdiğini, koltuk seçiminin toplam tutarı doğru hesapladığını ve bilet formunun konsola özet + onay logu bastığını doğruladıysanız, bu haftanın davranış katmanı tamamlanmış demektir.

Adım 20: Derlenebilirlik kapısını çalıştırın

? Ne yapacağız? istemci klasöründe sırasıyla npm ci (ya da npm install), npx tsc --noEmit ve npm run build komutlarını art arda çalıştıracaksınız.

i Neden? Bu üç komut, bu haftanın “kapısıdır” — npm ci bağımlılıkların package-lock.json'a birebir sadık kurulduğunu, tsc --noEmit yazdığınız tüm TypeScript kodunun (tipler, veri, DOM fonksiyonları) tutarlı olduğunu, npm run build ise projenin gerçek bir sunucuya yüklenebilecek durumda olduğunu garanti eder. Bir proje “çalışıyor” demek yetmez — bu üç komutun hatasız bitmesi, kodun teslim hazır olduğunun kanıtıdır.

>_ Terminal

```
$ npm ci
$ npx tsc --noEmit
$ npm run build
```

👁 Beklenen sonuç: Üç komut da hatasız biter; son komut dist/index.html ve dist/assets/ altında CSS/JS dosyalarını üretir.

Adım 20 (devamı)**Beklenen Çıktı**

```
vite v8.2.2 building client environment for production...
transforming...
built in 362ms
```

✂ Sorun Giderme

Belirti	Olası neden	Çözüm
node: command not found npm create vite@latest React şablonunu gösteriyor ama vanilla-ts listede yok npx tsc --noEmit "Can not find module './tipler.ts'" hatası veriyor	Node.js kurulu değil ya da path'e eklenmemiş Eski bir create-vite sürümü önbellege alınmış olabilir tipler.ts dosyası src klasöründe değil ya da dosya adı yanlış yazılmış	nodejs.org 'dan Node 24 LTS'i indirip kurun, terminali yeniden başlatın npm create vite@latest -- --template vanilla-ts ile şablonu doğrudan komut satırından belirtin Dosyanın tam olarak istemci/src/tipler.ts yolunda olduğunu ve içe aktarma satırındaki yolun (./tipler.ts) birebir eşleştiğini kontrol edin
Koltuğa tıklayınca hiçbir şey olmuyor, konsolda hata yok Bilet formunda "Gönder"e basınca sayfa yine de yenileniyor	koltukPlaniUret fonksiyonu main.ts'te hiç çağrılmamış ya da yanlış kapsayıcı elemana bağlanmış olay.preventDefault() çağırısı unutulmuş ya da submit yerine click dinleyicisi eklenmiş	main.ts'teki querySelector('#koltuk-plani') çağrısının index.html'deki id'yle birebir eşleştiğini doğrulayın Olay dinleyicisinin form elemanına 'submit' olayıyla eklendiğini ve dinleyicinin ilk satırının preventDefault() olduğunu kontrol edin
npm run build "Type ... is not assignable" hatasıyla duruyor	veri.ts'teki bir nesnenin bir alanı eksik ya da yanlış tip-te (ör. durum: 'bos' yerine durum: 'BOS') yazılmış	Hata mesajındaki satır numarasına gidip alan adını ve değerini tipler.ts'teki tanımla birebir karşılaştırın

5 • İzleme ➡ Aktarım — Başka Nerede?

Bugün Hafta 2'nin **HTML/CSS iskeletine tek satır dokunmadan** üstüne yazdığınız TypeScript — Etkinlik/Koltuk/ Bilet tipleri, dinamik kart üretimi, tıklanabilir koltuklar, gerçek bir form gönderimi — Etkinlik/Bilet Sisteminin ilk davranış katmanı olarak dönem boyunca sizinle kalacak. Hf. 4'te her koltuğa elle addEventListener eklemenin ve listeyi elle createElement ile üretmenin koltuk/etkinlik sayısı arttıkça nasıl hızla karmaşılaştığını göreceksiniz — React bu güncellemeyi sizin yerinize otomatikleştirecek; bugünkü Etkinlik/ Koltuk/Bilet tipleri ve veri.ts'teki sabit diziler değişmeden, React bileşenlerinin props tipleri olarak kullanılmaya devam edecek. Bugün öğrendiğiniz değer/referans ayrımı, Hf. 4'te React'ın state güncellemelerinde "mevcut diziyi elle değiştirmek" ile "yeni bir dizi oluşturmak" arasındaki kritik farkın temelini oluşturacak. Hf. 6'da bugün simüle ettiğiniz setTimeout+Promise yerini FastAPI'ye gerçek bir ağ isteğine bırakacak — async/await söz dizimi aynen kalacak. Son olarak Hf. 14'te bugün kurduğunuz npm run build çıktısı (dist/) bir Docker imajının içine konup nginx ile

gerçek kullanıcılara sunulacaktır.